

IT603: Introduction to Programming
Autumn 2025-26
Final Project

Project Description:

The Snake Game is a classic arcade game where the player controls a snake that grows in size as it consumes food. The game ends if the snake collides with itself or the boundaries of the play area.

Requirements:

- Functional Requirements:
 - Game Setup:
 - i. A grid-based game area (e.g., NxN cells).
 - ii. Snake starts as a line with 3 cells in length.
 - iii. Food randomly appears in the grid.
 - Gameplay Mechanics:
 - i. The snake moves continuously in the current direction.
 - ii. Players can control the direction using keyboard inputs (e.g., W/A/S/D or arrow keys).
 - iii. Eating food increases the snake's length by one cell.
 - iv. The score increases with each piece of food consumed.
 - Game Over Conditions:
 - i. The game ends if the snake collides with itself or the boundary.
 - ii. Display the final score upon game over.
 - Score Display:
 - i. Display the current score during the game.
 - ii. Allow restarting the game or exiting after a game over.
- Non-Functional Requirements:

- Ease of Use:
 - i. The game interface should be simple and easy to navigate.
 - ii. Instructions should be clear for beginners.
- Performance:
 - i. The game should run smoothly without significant lag.
- Extensibility:
 - i. Code should be modular, allowing future enhancements like obstacles or multiple levels.

Technical Details:

● Programming Concepts to Use:

- OOPS Concepts:
 - i. Class and Object: Represent the Snake, Food, and GameBoard as classes.
 - ii. Encapsulation: Manage game logic within appropriate classes.
 - iii. Inheritance (optional): For extending functionality, like different types of snakes or food.
 - iv. Polymorphism (optional): For handling varied behaviors like normal and special food.
- Data Structures(suggested. Can be different):
 - i. **Queue or Linked List:** To represent the snake's body.
 - ii. **Array/multi dimensional array:** For the grid.
- Basic Concepts:
 - i. Loops and Conditionals.
 - ii. Functions and modular design.

Acceptance Criteria:

- The game starts and runs without crashes or errors.
- The snake moves as per the player's input.
- The snake grows upon eating food, and the score updates correctly.
 - Unfair Food Placement: Ensure food does not spawn on the snake's body.
 - The game detects collisions (with itself and boundaries) and ends appropriately.
 - The final score and highest score is displayed on the game-over screen. option to restart is also provided

Evaluation Criteria:

1. Project completion with acceptance criteria: 30% [Make it work]
2. Viva(30%):
 - a. Through explanation of [Make it efficient]
 - i. approach taken
 - ii. Data structures used
 - iii. Tradeoffs made
3. UI/UX: 20% [Make it elegant]
4. Extensibility of code(15%):
 - a. Is your code receptive to new client requirements?
5. Continuous development(5%)
 - a. How regular you were in developing the solution during labs.

Note to Students:

1. You are encouraged to approach the Teaching Assistants or

Instructor with any doubts or difficulties encountered during the project. The focus is on learning and applying concepts effectively. (Refer point 5 of evaluation criteria)

2. Project is group wise. In Viva you are on your own. Anybody can be asked about any part of the project irrespective of whether you built that part or not.
3. Copy pasting might get you 30%(refer point 1 of evaluation criteria). Remaining 70% will require focus, thoroughness and critical thinking.